Ch3_Essential Enterprise Development Technologies

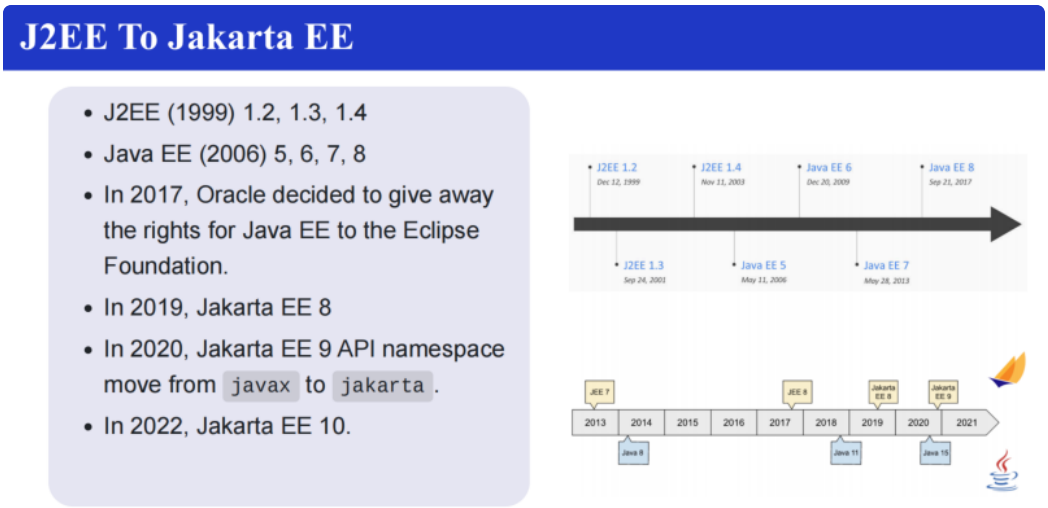
Introduction

J2EE → Java EE → Jakarta EE

它们并不是三个完全不同的平台，而是同一套企业级 Java 技术在不同历史阶段的名称

全称：**Java Enterprise Edition**

现在已经改名为：**Jakarta EE**



Java EE 是什么？ / What is Java EE

它不是一种新的编程语言，而是建立在 Java 之上的一套**企业级应用开发规范和技术体系**，主要用于开发大型 Web 系统和企业系统

Aim/ Specification/ Implementation

- The aim of the Java EE platform is to provide developers with a powerful set of API while shortening development time, reducing application complexity, and improving application performance.
- Specifications: Java EE API describes how an enterprise application should be created, what components should contain, etc

- Implementations (Application Servers) : GlassFish, Payara,WeblogicServer, Jboss etc

Java EE 与 Java SE

- The Java EE platform is built on top of the Java SE platform.
- The Java EE platform provides APIs and runtime environment for developing and running
  - large-scale
  - multi-tiered
  - scalable
  - Reliable
  - secure network applications

Java SE 提供基础能力，例如：

- Java 语法
- JVM
- 集合框架
- 异常处理
- 文件 I/O
- 网络基础
- 多线程基础

Java EE 在 Java SE 之上增加企业开发能力，例如：

- Web 请求处理
- 数据库持久化
- 事务管理
- 身份认证
- 消息通信
- 依赖注入
- Web Service

> **Java EE = Java SE 基础 + 企业级 API 和运行环境**
>
> **Java EE 是建立在 Java SE 之上的企业级 API 规范集合**

Java EE 的特点

- **Set of interoperable API specifications**
- **Supports development of enterprise grade systems**
- **APIs governed by the Java Community Process (JCP)**
- **Multiple open-source and proprietary implementations**

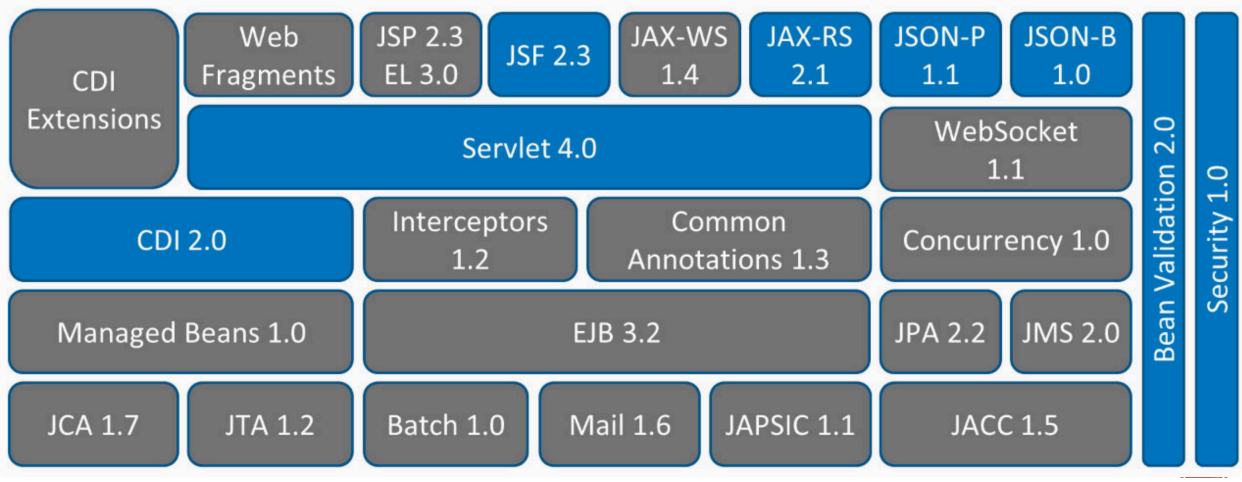
a. Set of interoperable API specifications = 一组标准

> interoperable 尤指计算机系统或软件、不同机器等）可互操作的，可共同操作的，能交换和利用信息的

Java EE 不是一个具体软件，也不是单独的编程语言，而是一组能够互相配合的 **API 规范**。

“规范”规定了：

- API 应该提供哪些类和方法
- 应用服务器应该如何执行这些 API
- 不同组件应该如何协作
- 开发者编写的程序应该得到什么结果



Web 开发

- **Servlet**：处理 HTTP 请求与响应
- **JSP**：动态生成网页
- **JSF**：基于组件构建 Web 用户界面
- **WebSocket**：客户端与服务器之间进行双向实时通信

数据与事务

- **JPA**：将 Java 对象映射到数据库表
- **JTA**：管理事务
- **JCA**：连接企业信息系统

业务组件与依赖管理

- **EJB**：构建企业业务组件，支持事务、安全等功能
- **CDI**：依赖注入和对象生命周期管理
- **Managed Beans**：由容器管理的 Java 对象
- **Interceptors**：在方法执行前后加入通用处理

消息与后台任务

- **JMS**：消息通信
- **Batch**：批处理任务
- **Concurrency**：容器管理的并发任务
- **Mail**：发送电子邮件

安全与验证

- **Bean Validation**：验证对象数据
- **Security**：身份认证与权限控制
- **JASPIC、JACC**：更底层的安全规范

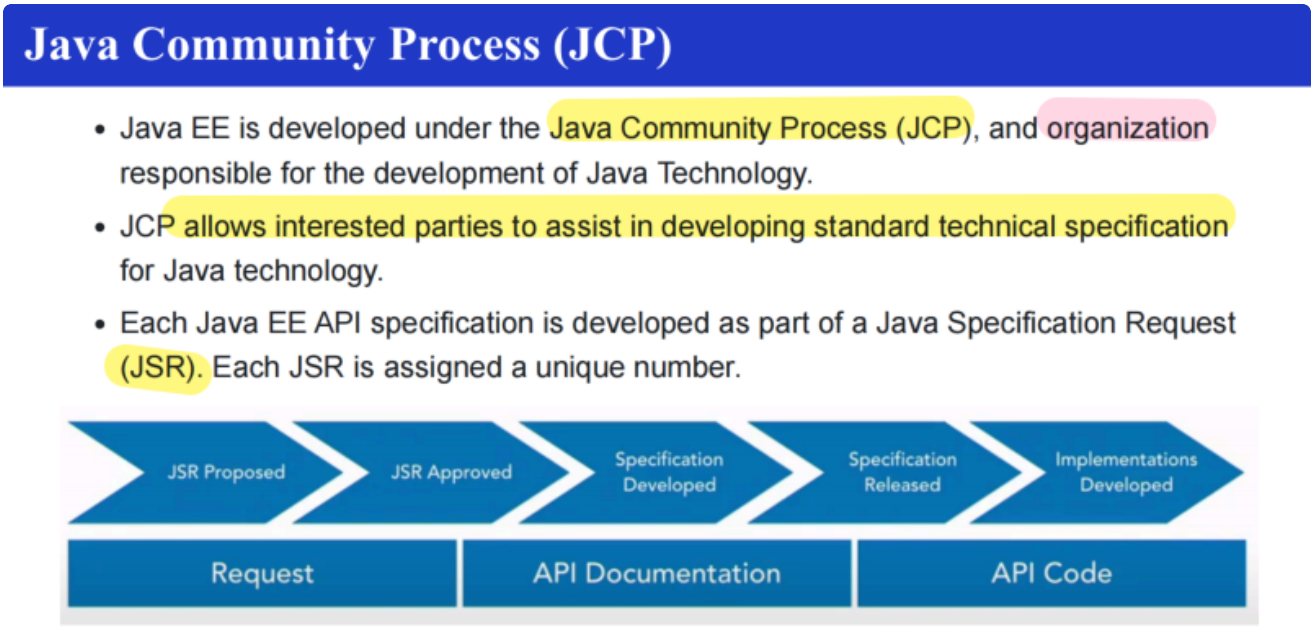
Web Service 与数据格式

- **JAX-RS**：创建 RESTful Web API
- **JAX-WS**：创建 SOAP Web Service
- **JSON-P**：处理 JSON 数据
- **JSON-B**：在 Java 对象和 JSON 之间转换

b. Supports development of enterprise grade systems

Java EE 提供相关 API 和运行环境，开发者不用从头实现基础功能，如支持大量用户，同时处理多个请求，连接数据库，管理事务，保证安全，在多台服务器上运行，长期稳定运行

c. APIs governed by the Java Community Process (JCP)



JSR 是规范制定过程，不是普通的 Java 程序库：每一项 Java 规范通常通过一份 JSR 提出，每个 JSR 都有唯一编号，用于标识某项具体规范

1. **JSR Proposed**：有人提出新的 Java 技术规范或修改现有规范。
2. **JSR Approved**：提案经过审核，同意进入开发阶段。
3. **Specification Developed**，专家组设计 API、行为规则和相关文档。
4. **Specification Released**，正式发布规范
5. **Implementations Developed** 厂商和开源社区根据规范开发实际产品。

d. Multiple open-source and proprietary implementations 同一套 Java EE 规范可以有多个实现。

例如，规范规定 Servlet 应该如何工作，而不同应用服务器负责提供实际实现

- Java EE specification = 建筑设计标准
- Application server = 按照标准建造的建筑

只要服务器符合 Java EE 规范，理论上，同一个 Java EE 应用就可以在不同服务器之间迁移。 而这就是 portability，即可移植性的重要来源

Java EE 为什么会出现?

- We are in the situation of developing a **complex, large-scale, portable, scalable, reliable, secure, transactional, distributed** system. **企业系统常见的要求**

  - **Complex**：系统功能和业务逻辑复杂
  - **Large-scale**：系统规模大、用户和数据量多
  - **Portable**：可以在不同平台或服务器上运行
  - **Scalable**：用户增加时，系统可以通过增加服务器扩展
  - **Reliable**：系统需要稳定运行，不能经常崩溃
  - **Secure**：必须提供身份认证、权限控制和数据保护
  - **Transactional**：多个操作需要作为一个整体成功或失败
  - **Distributed**：系统的不同部分可能运行在不同计算机上

  例如银行转账包含两个操作：

  $$\text{账户 A 扣款} + \text{账户 B 入账}$$

  这两个操作必须同时成功。如果账户 A 已扣款，但账户 B 没有收到钱，系统就出现了严重错误。因此，它们需要放在同一个 **transaction（事务）** 中：

  $$\text{全部成功} \quad \text{或} \quad \text{全部失败并回滚}$$

- Who is the customer? **需要大型信息系统的组织：用户多、业务复杂**
  - A HUGE bank, a chain of hypermarkets, a Fortune 500 company, etc.
- What do we know?
  - Programming language: Java, C++, python, JavaScript ...
  - System Design Technologies: Design patterns, enterprise system architectures, OOT (OOA/OOD).
  - Various protocols: TCP, UDP, HTTP, SOAP, etc.
- What do we want? **即使开发人员懂编程、架构和网络协议，从头实现企业系统仍然很困难**
  - A framework that will make our live as easy as possible.

需要一个框架替开发人员处理通用的底层工作。Java EE/Jakarta EE 正是这样的标准平台，让开发者把主要精力放在业务逻辑上，而不是重复编写数据库连接、事务和安全管理代码

> 企业系统非常复杂 -> 仅有编程语言和设计知识不够方便 -> 需要统一的企业开发框架 ->Java EE / Jakarta EE

Advantages of Using Java EE in Enterprise

- **Java can run on multi-platform (WORA)**
- **Object Oriented design for system architecture**
- **Plenty of framework for system building**
- **Wide dissemination with minimal cost**
- **Dominating the e-Commerce industry**
- **FOSS (free and open source software)**

1. Java can run on multi-platform (WORA)

Java 可以在多个操作系统上运行。WORA 是：Write Once, Run Anywhere， 即“编写一次，到处运行”

Java 源代码不会直接编译成特定操作系统的机器码，而是先编译成字节码：

```
Plain Text
```

```
Java source code (.java)
          ↓ javac
Java bytecode (.class)
          ↓ JVM
Windows / Linux / macOS
```

只要操作系统安装了相应的 JVM，同一份 `.class` 文件通常不需要重新编译即可运行

2. Object Oriented design for system architecture

Java 是一种面向对象语言，适合使用类和对象组织大型系统

例如，一个电商系统可以分成：Customer, Product, ShoppingCart, Order...

每个类负责一部分**数据和行为**。这种方式有助于：

- 将大型系统拆成较小模块
- 降低不同模块之间的依赖

3. Plenty of framework for system building

Java 拥有大量成熟框架和工具，例如：

- Spring：依赖注入和企业应用开发

- Spring Boot：简化应用配置和部署
- Hibernate：数据库对象关系映射
- Jakarta EE：企业级 API 规范
- Maven、Gradle：依赖和项目构建管理
- JUnit：自动化测试

这些框架解决了许多重复问题，使开发者可以把更多精力放在业务逻辑上。

4. Wide dissemination with minimal cost

Java 传播范围广，而且使用成本相对较低。

原因包括：

- Java 开发工具容易获得
- OpenJDK 可以免费使用
- 有大量免费的框架和程序库
- 能在普通服务器和主流操作系统上运行
- 社区规模大，学习资料丰富

对于企业而言，可以使用开源 Java 生态建立系统，而不一定要购买昂贵的专有开发工具

5. Dominating the e-Commerce industry

在电子商务和大型企业后端系统中曾占有重要地位。

Java 的并发支持、企业框架、事务管理、安全机制和成熟生态适合这些需求。因此，许多大型电商、银行和企业后台长期使用 Java。

6. FOSS (free and open source software)

即“自由及开源软件”。

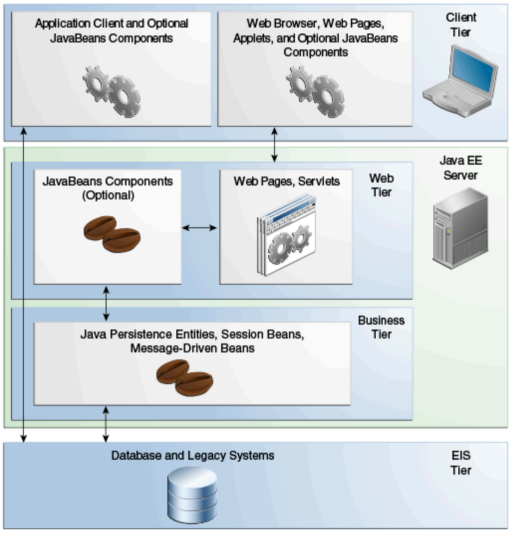
- **Free** 更强调使用、研究、修改和分发软件的自由，不一定只表示价格免费。
- **Open Source** 表示软件源代码开放，并允许按照许可证使用和修改。

Java EE application

- A Java EE component is a self-contained functional software unit that is assembled into an application at a specific tier and communicates with other components.

应用被划分成**多个层（Tier）**->每一层包含**不同组件（Component）**-> **组件由对应容器（Container）管理和运行**

- The Java EE platform uses a **distributed multi-tiered application model** for enterprise applications.
- Application logic is divided into **components** according to function.
- The application components that make up a Java EE application are installed on various machines.



Java EE 使用分布式多层应用模型 Distributed multi-tier application model：

- **Distributed**：组件可以分布在不同机器上
- **Multi-tier**：系统根据功能划分为多个层

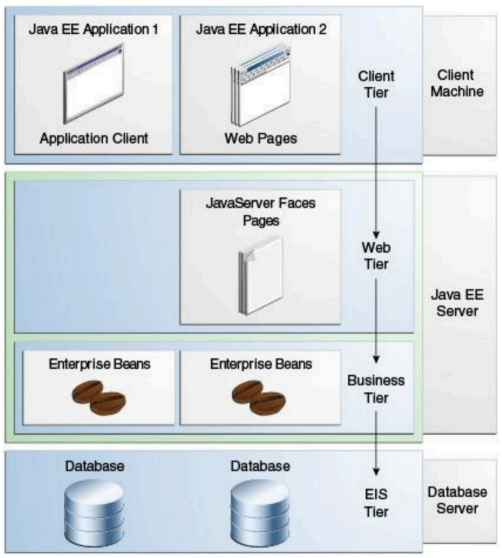
多层设计的主要目的是**职责分离**。用户界面、请求处理、业务规则和数据存储由不同部分负责，而不是把全部代码写在一起。

Java EE 的四层架构

In a class web-based Java EE application architecture, there are four tiers:

1. Client Tier
2. Web Tier
3. Business Logic Tier
4. Enterprise Information System (EIS) Tier.

- Java EE applications are made up of ***components***



第一层：Client Tier

Client Tier 是用户直接接触的**前端**。

常见组件包括：Web pages, application client, 早期的 Java applet

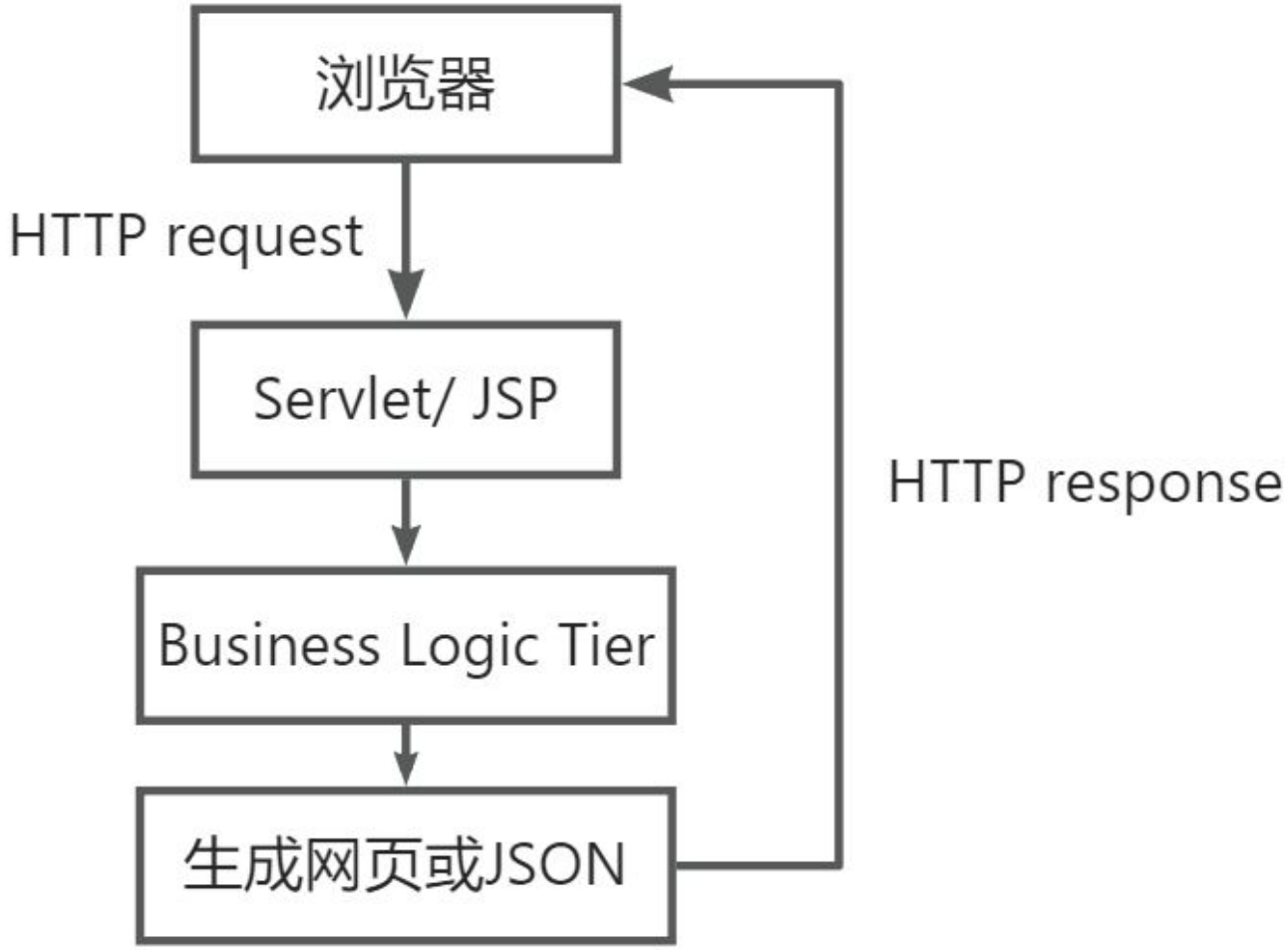
它负责：

- 接收用户输入
- 向服务器发送请求
- 显示服务器返回的结果

第二层：Web Tier

- **Web Tier**
  - **Java servlet**: Java servlets are server side modules, typically used to process a request and send back response in the web applications.
  - **Java Server Pages (JSP)**: JSPs are also server side modules used to process the web requests. JSPs (Java Server Pages) are great for handling requests that generate large HTML markup responses.
  - **Java Server Faces (JSF)**: JSF makes creating user interface on the server side modular by incorporating the MVC design pattern in its implementation.

Web Tier 接收客户端请求，**调用业务逻辑，并生成响应**。



a. Servlet

Servlet 是运行在**服务器端的 Java 组件，用来处理 HTTP 请求和响应**。

Servlet 通常负责：

- 读取请求参数
- 调用业务服务
- 控制请求流程
- 返回响应

它不应该包含大量核心业务逻辑。

b. JSP

JSP 全称是 JavaServer Pages，主要用于**生成动态 HTML**。

服务器执行 JSP 后，会生成普通 HTML 并发送给浏览器。

c. JSF

JSF 全称是 JavaServer Faces，是一种服务器端 UI 组件框架。

它提供按钮、输入框、表单验证和页面导航等组件，并常用于 MVC 架构。

MVC 是：

- **Model**：数据和业务状态
- **View**：用户界面
- **Controller**：处理请求并协调 Model 和 View

第三层：Business Logic Tier

- **Business Logic Tier**
  - **Enterprise Java Beans (EJB)**: are the Java classes where you can write your business logic.

Business Logic Tier 负责真正的业务规则。

第四层：Enterprise Information System Tier

- **Enterprise Information System (EIS) Tier**
  - **Java Database Connectivity (JDBC)**: JDBC is a specification to access relational database in a common and consistent way.
  - **The Java Persistent API (JPA)**: JPA gives you the flexibility to map the objects to the tables in relational database and execute the queries with or without using Structured Query Language (SQL).
  - **Web Services**: Web services are remote application components that expose self-contained APIs.
    - Simple Object Access Protocol(SOAP)
    - Representational State Transfer (REST).

它表示企业应用所连接的后端信息系统

a. JDBC

它允许 Java 程序直接执行 SQL：

JDBC 接近数据库底层，开发者通常需要自己：

- 编写 SQL
- 读取查询结果

- 将结果转换成 Java 对象
- 处理连接和异常

b. JPA

它用于把 Java 对象**映射**为数据库表，这种机制称为 ORM：Object-Relational Mapping}

JDBC 与 JPA 的关系可以简单理解为：

```
JPA：较高层的对象操作方式
 ↓
JDBC：较底层的数据库访问方式
 ↓
关系型数据库
```

c. Web Services

Web Service 允许系统通过网络调用另一个系统提供的功能。

i. SOAP

SOAP 通常使用 XML，具有较**严格的消息格式和服务契约**。

```
系统A → SOAP XML消息 → 系统B
```

它在传统企业系统、政府系统和银行系统中较常见。

ii. REST

REST 通常使用 HTTP 方法：

- `GET`：获取数据
- `POST`：创建数据
- `PUT`：更新数据
- `DELETE`：删除数据

Containers

组件本身只包含应用逻辑，但它还需要许多基础功能才能运行

> **Containers are the interface between components and the low-level platform-specific functionality that supports the component.**
>
> - **Before a web, enterprise bean, or application client component can be executed, it must be assembled into a Java EE module and deployed into its container"**
> - **Web/servlet Containers** manage the execution of web pages, Java servlets, JSP, JSF etc. Implemented in most application servers, such as Tomcat.
> - **Java EE/EJB containers** manages the execution of EJB, JMS, etc. and are implemented in the "heavy" Java EE servers, such as ***Glassfish***.

Component 写业务逻辑，Container 提供运行环境

- Web Container，管理 Web 组件
  - 例如，Servlet，JSP，JSF，WebSocket
- EJB Container，管理 EJB 等企业业务组件。

Java EE server & Web server

Web server 是什么?

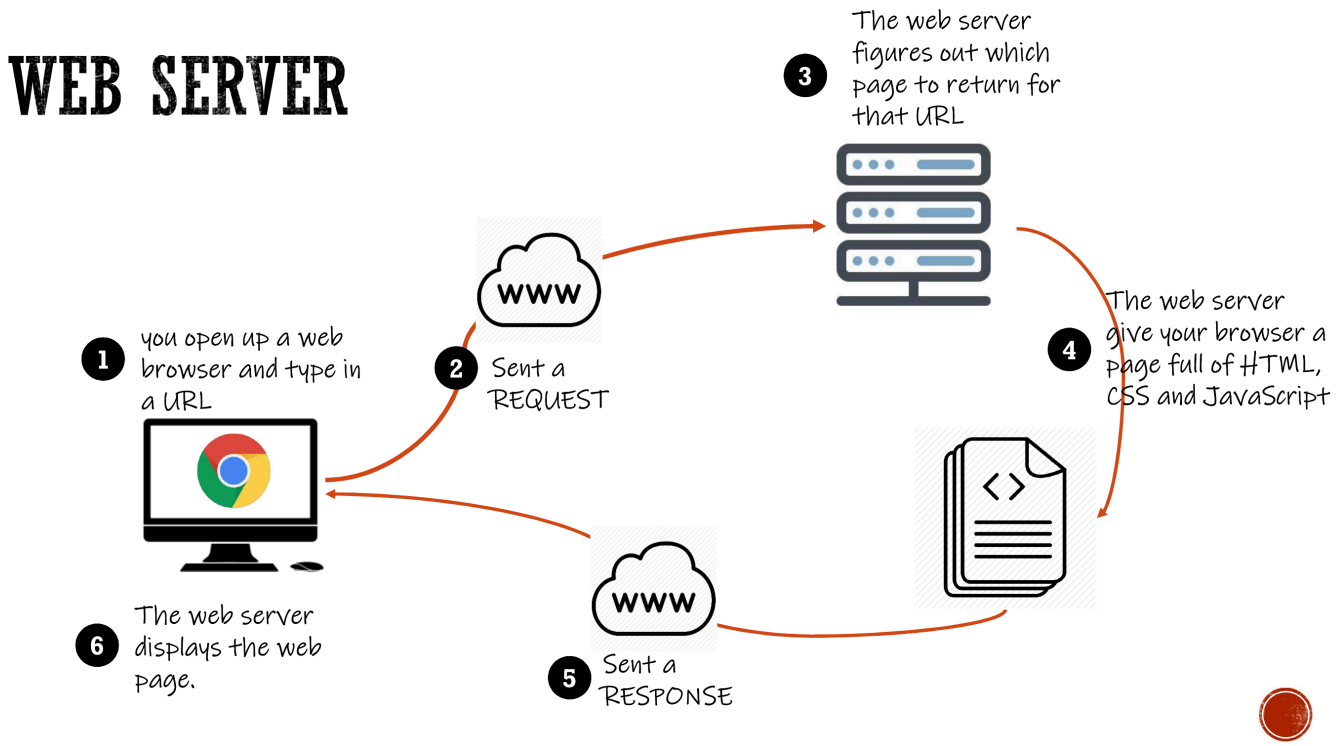
Web server 是通过 HTTP 向客户端提供 Web 内容的软件。

> **A web server is software that helps to deliver web content (web pages) to the client (e.g. web browser) through the Internet using HTTP protocol.**
>
> - Handle static web contents with **HTTP protocol**
> - It processes the HTTP Request and Response to deliver **static web contents** (e.g. web pages, images, documents, etc.).
> - It doesn't process the server-side programs (PHP, .NET, Java, etc.) to deliver dynamic web contents
> - Use **port 80** to provide services in general
> - it reserves this port to provide HTTP services as default.
> - Once it is started, you cannot run other services to use the same port.

1. **静态内容：**Web server 最擅长处理静态内容，也就是服务器上已经存在的文件，如 web pages, images, documents，Web server 会读取文件并返回。

2. **动态内容**： Web server 本身通常**不负责执行**复杂的**服务器端程序**， **动态页面**需要应用程序**根据**用户、数据库或**请求参数生成**。因此通常需要 Web Server 把**请求转交给后端应用**，或者由 Application Server 处理。

浏览器访问网页的过程



常见 Web Server

February 2021

Below are the latest statistics of the market share of all sites of the top web servers on the Internet by Netcraft February 2021 Web Server Survey

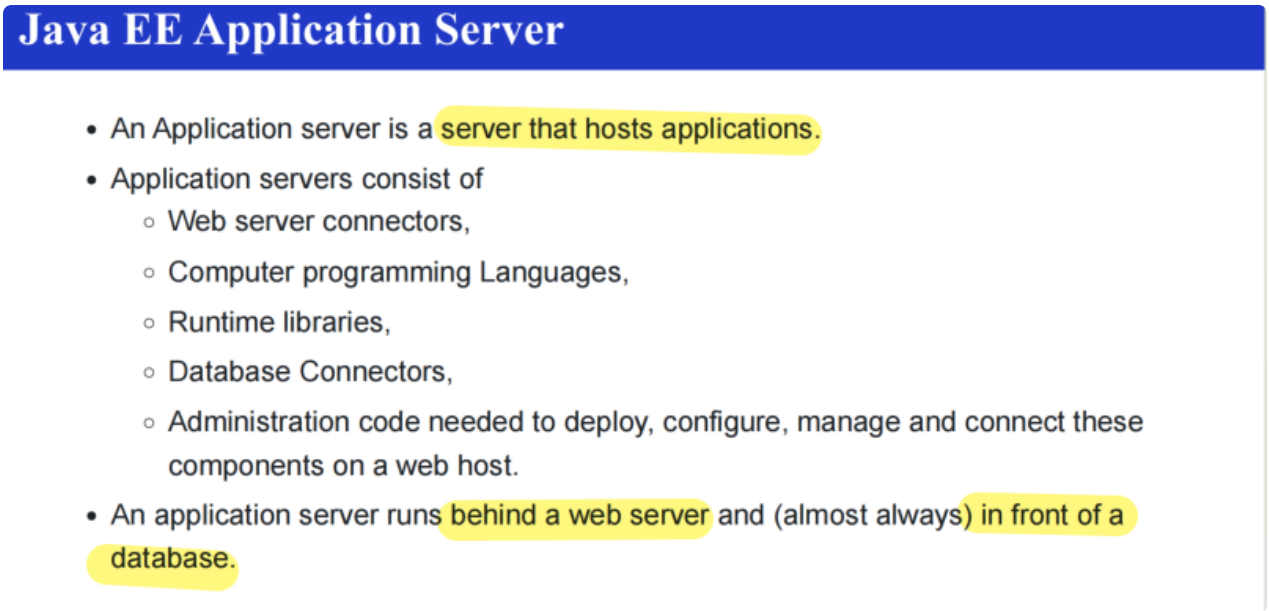
Product	Vendor	Percent
nginx	NGINX, Inc.	34.54%
Apache	Apache	26.32%
IIS	Microsoft	6.5%
OpenResty	OpenResty Software Foundation	6.36%
Cloudflare Server	Cloudflare, Inc.	5.0%

https://en.wikipedia.org/wiki/Web_server#Market_share



Application Server 是什么？

Application server 是能够托管和运行应用程序的服务器软件。



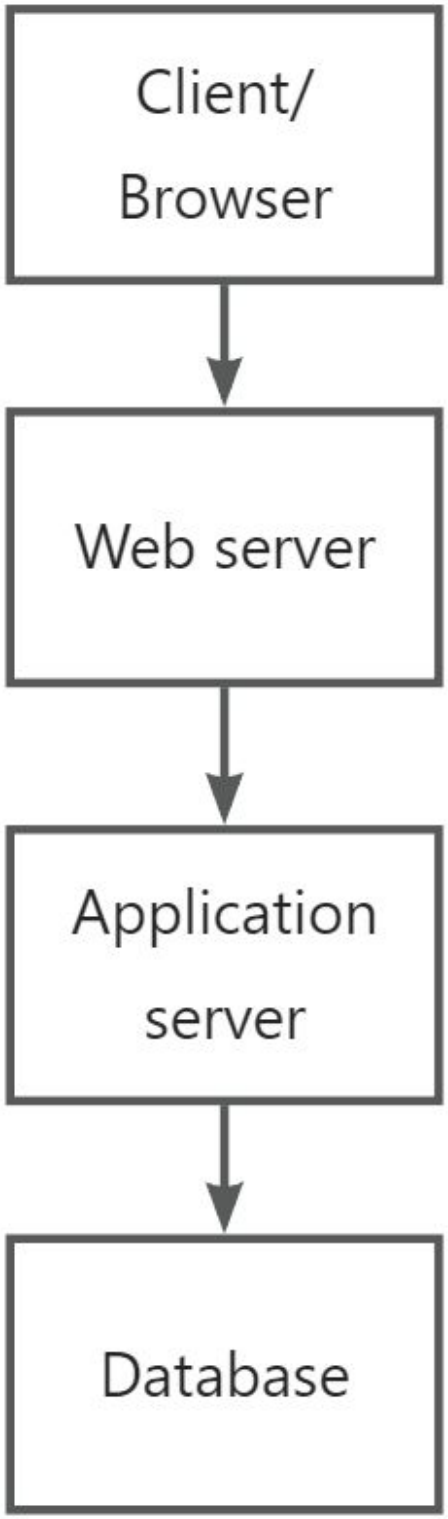
Web Server 主要负责：

```
1  接收请求 → 返回文件
```

Application Server 还可以负责：

Plain Text

```
接收请求
→ 执行业务逻辑
→ 访问数据库
→ 管理事务
→ 处理安全
→ 返回动态结果
```



浏览器

Nginx

Java EE Application Server

MySQL / PostgreSQL

Web Server 的作用

- 接收外部 HTTP 请求；
- 提供**静态文件**；
- 作为反向代理；
- 过滤和**转发**请求；
- 处理 HTTPS；
- 进行负载均衡。

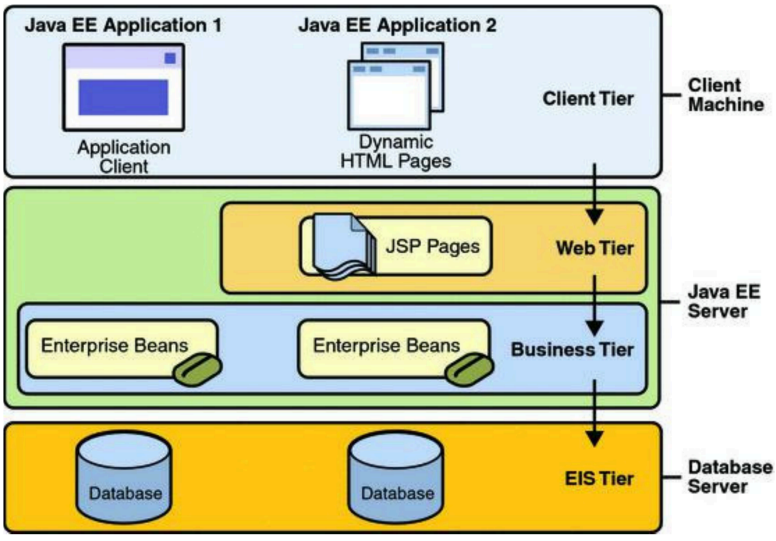
Application Server 的作用

- 执行后端应用；
- 处理业务逻辑；
- 管理事务；
- 管理数据库连接；
- 处理用户权限；
- 运行 Java EE 组件。

Java EE Application Server

Java EE 定义了企业应用服务器应该支持的一组 API 和功能。

- Java EE defines the core set of API and features of Java Application Servers.
- A Java EE application server contains an EJB container as well as a servlet container and implements all the required specifications of Java EE.



Web Container

Web Container 管理：

- **Servlet**
- JSP
- JSF
- WebSocket

它负责处理 Web 请求、Servlet 生命周期和 Session 等内容

EJB Container

EJB Container 管理 Enterprise JavaBeans，负责：

- 业务逻辑组件
- 事务
- 安全
- 并发
- 生命周期
- 依赖注入
- 远程调用

GlassFish

GlassFish 是一个 Java EE Application Server 实现。

- **Glassfish** is an open-source and commercial full Java EE application server implementation.
- It provides all the features in the Java EE specification, including a web container and an EJB container.
- It will be used throughout this course.



OTHER JAVA APPLICATION SERVER

- **Apache Tomcat** is the most common and popular web container
  http://tomcat.apache.org/tomcat-8.5-doc/index.html
  - http://tomcat.apache.org/download-85.cgi



- **WildFly/JBoss AS** is an open source and free full Java EE application server implementation.







项目	Web Server	Web Container	Application Server
主要任务	返回静态网页	运行 Web 组件	运行完整企业应用
典型内容	HTML、CSS、图片	Servlet、JSP	Servlet、EJB、JPA、JMS 等
业务逻辑	通常很少	可以调用业务层	可以完整执行
事务管理	通常不负责	有限	支持
数据库连接	通常通过代理	可以连接	提供完整管理
例子	Nginx、Apache、IIS	Tomcat	GlassFish、WildFly、WebLogic

Java EE development tools

Development Tool Chain

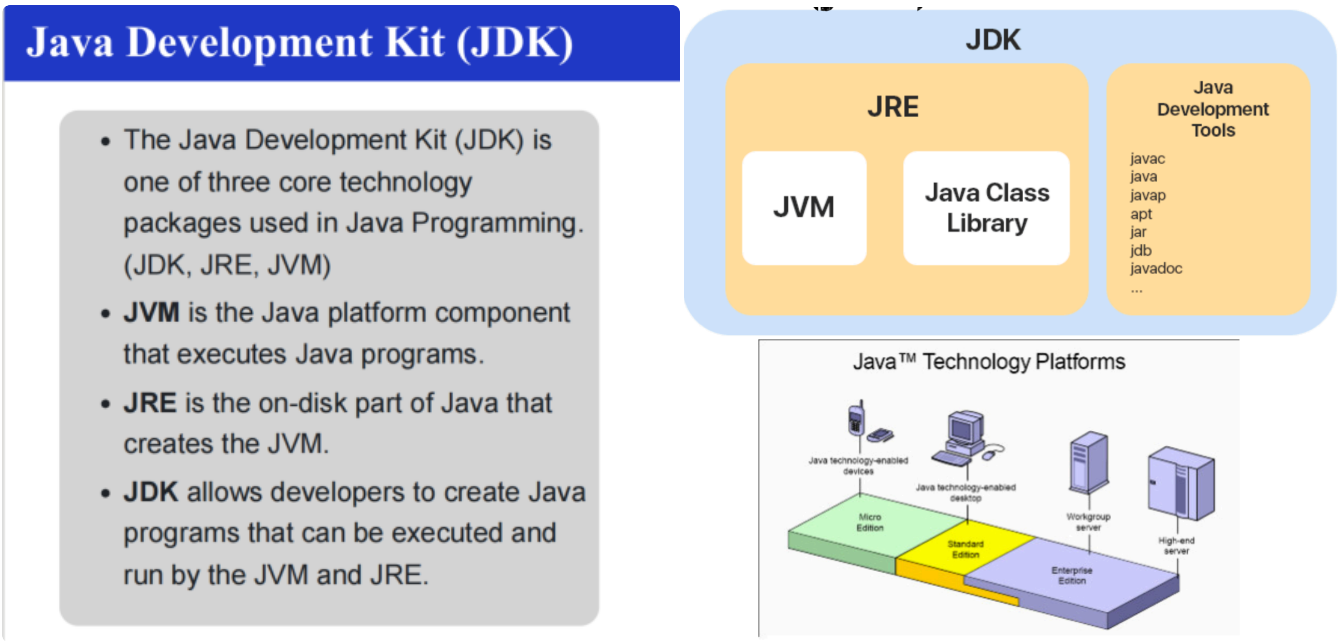
Java EE 开发通常需要一组工具，而不是只安装一个 Java 程序。

典型工具链是：

```
JDK 编译和运行 Java 程序；
 ↓
IDE 编写、调试和管理代码；
 ↓
Build Automation  自动编译、打包和测试；
 ↓
Test Automation 自动运行测试；
 ↓
Java EE Server 部署和运行企业应用。
```

1. Software Development Kit (SDK) & JDK

- **Software Development Kit (SDK)**: A SDK is a collection of software development tools in one installable package
  - JDK (Java Development Kit)



a. JVM

Java Virtual Machine，JVM 负责执行 Java 字节码。

Java 程序的运行过程是：

Plain Text

```
.java 源代码
    ↓ javac
.class 字节码
    ↓ JVM
程序运行
```

JVM 是**跨平台的关键**。不同操作系统有不同版本的 JVM，但它们都可以执行相同的 Java 字节码。

b. JRE

Java **Runtime** Environment

JRE 主要包含：

- **JVM；**
- Java 类库；
- **运行** Java 程序所需的其他文件。

它适合运行 Java 程序，但通常不包含完整的开发工具。

c. JDK

Java Development Kit，JDK = **JRE** + **Java开发工具**

常见工具包括：

- `javac`：Java 编译器；
- `java`：运行 Java 程序；
- `jar`：打包 JAR 文件；
- `javadoc`：生成 API 文档；
- `jdb`：调试程序；
- `jlink`：创建运行时镜像。

组件	**作用**
JVM	执行字节码
JRE	提供**运行**环境
JDK	提供**开发和运行**环境

通常：

- 只运行程序：需要运行环境；
- 编写和编译程序：需要 JDK。

现在的 Java 发行版通常主要安装 JDK

2. IDE

IDE 把很多开发功能集中在一个图形界面中

IDE 的作用主要是提高开发效率，并不是 Java 程序运行所必需的。

- **Integrated Development Environment (IDE)**: An IDE is a software for building applications that combines common developer tools into a single graphical user interface (GUI).
  - Eclipse, NetBeans, etc.

没有 IDE 也可以使用命令行开发 Java：

```
Plain Text
```

```
1  javac Hello.java
2  java Hello
```

3. Build Automation

Build Automation 是构建自动化。

- **Build Automation**: Build automation is a process of automating the creation of a software build and the associated processes including: compiling, packaging and running automated tests.
  - Apache Maven, Gradle

它把重复的项目操作自动执行，如果手动执行，每次都要输入很多命令。构建工具可以把这些步骤写进配置文件，然后通过一条命令完成。

4. Test Automation

测试自动化是使用**独立的软件**自动执行测，并比较：实际结果 vs 预期结果

- **Test Automation**: Test automation is the use of software separate from the software being tested to control the execution of tests and the comparison of actual outcomes with predicted outcomes.
    - Junit

Standalone Java Application 和 Java Enterprise Application

Standalone Java Application	Java Enterprise Application
Run on local machine	Run on Java EE Server
Main function as entry point	Without main function
Compile to `class` or `jar` file	Package to `war` or `ear` file format

Standalone Java Application

独立 Java 程序通常：

- 在本地机器上运行；
- 有 `main()` 方法作为入口；
- 编译成 `.class` 或 `.jar` 文件；
- 由 JVM 直接启动。

Java Enterprise Application

Java EE 应用通常：

- 运行在 Java EE/Jakarta EE Server 上；
- 不依赖普通的 `main()` 方法作为应用入口；
- 包装成 `.war` 或 `.ear` 文件；
- 由 Application Server 负责启动和管理。

Summary

This chapter covered:

- Introduction of Java Enterprises Edition (J2EE, Java EE, Jakarta EE).
- Basic concepts about JCP and JSR
- Java EE APIs (JSF, Servlet, CDI, JAS-RS, JSON-P ...)
- Java EE Applications' components.
- Java EE Containers, Java EE application server, web server)
- Some popular Java EE development tools.